CS 412 / 512
Introduction to Computer Graphics

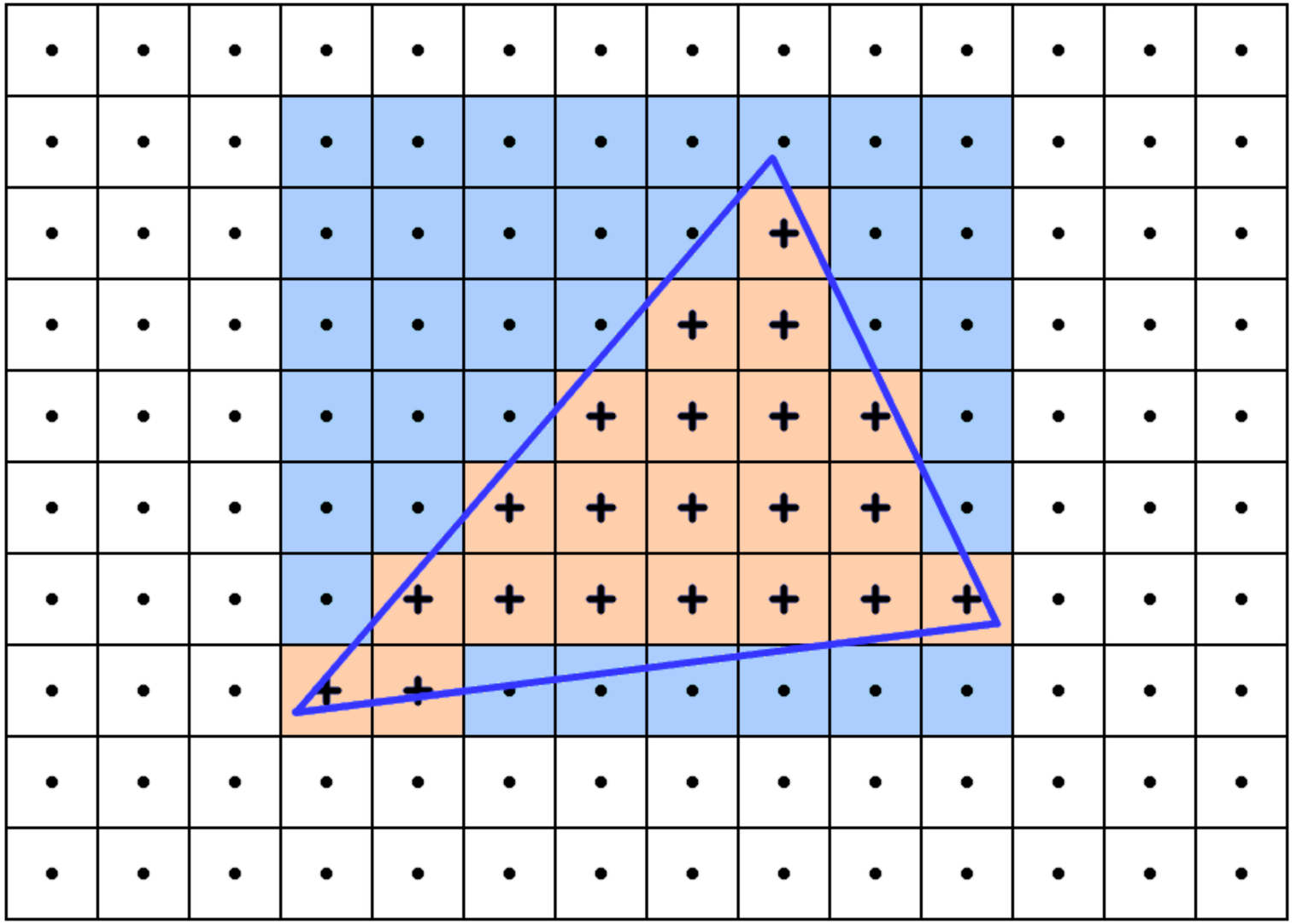
Zhu Wang
Assistant Professor of Computer Science



COMPUTER SCIENCE

Rasterization

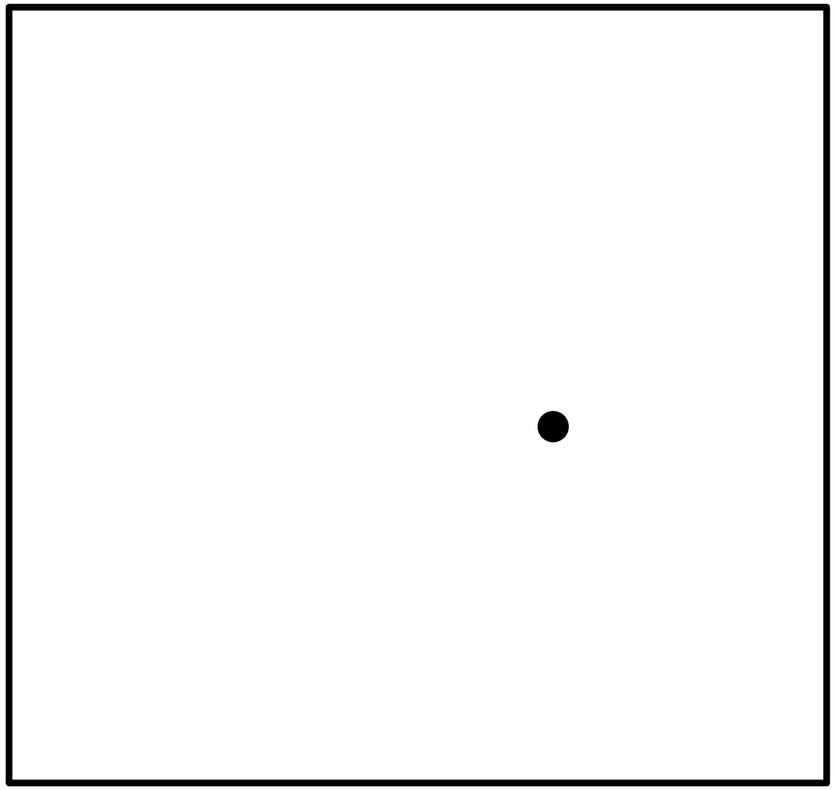
Rasterization



[source:Lisyarus Blog]

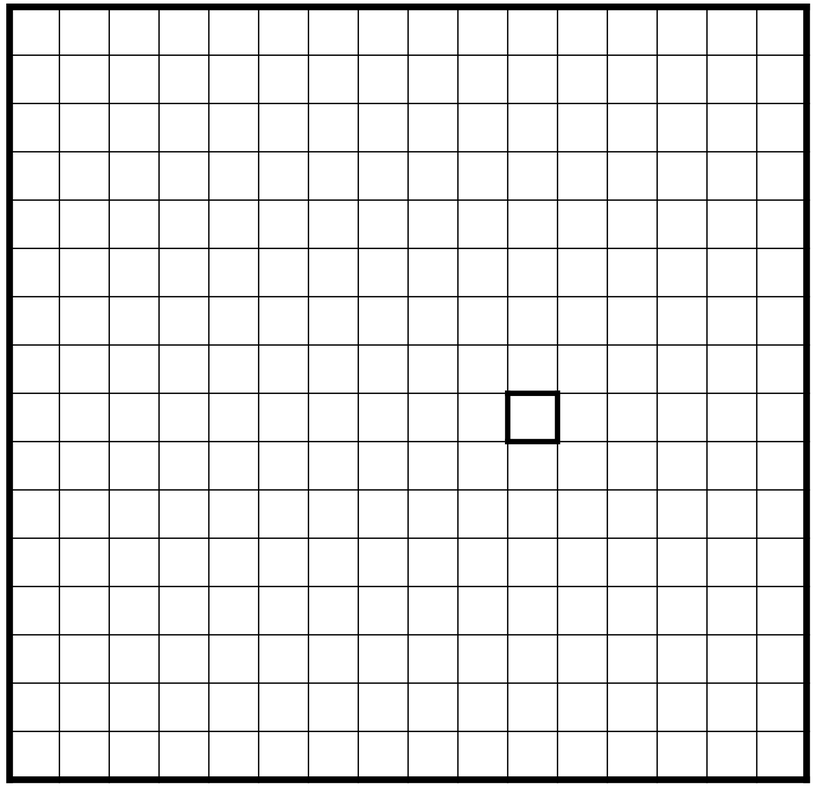
Continuous Space vs Discrete Pixels

(1.0, 1.0)



(-1.0, -1.0)

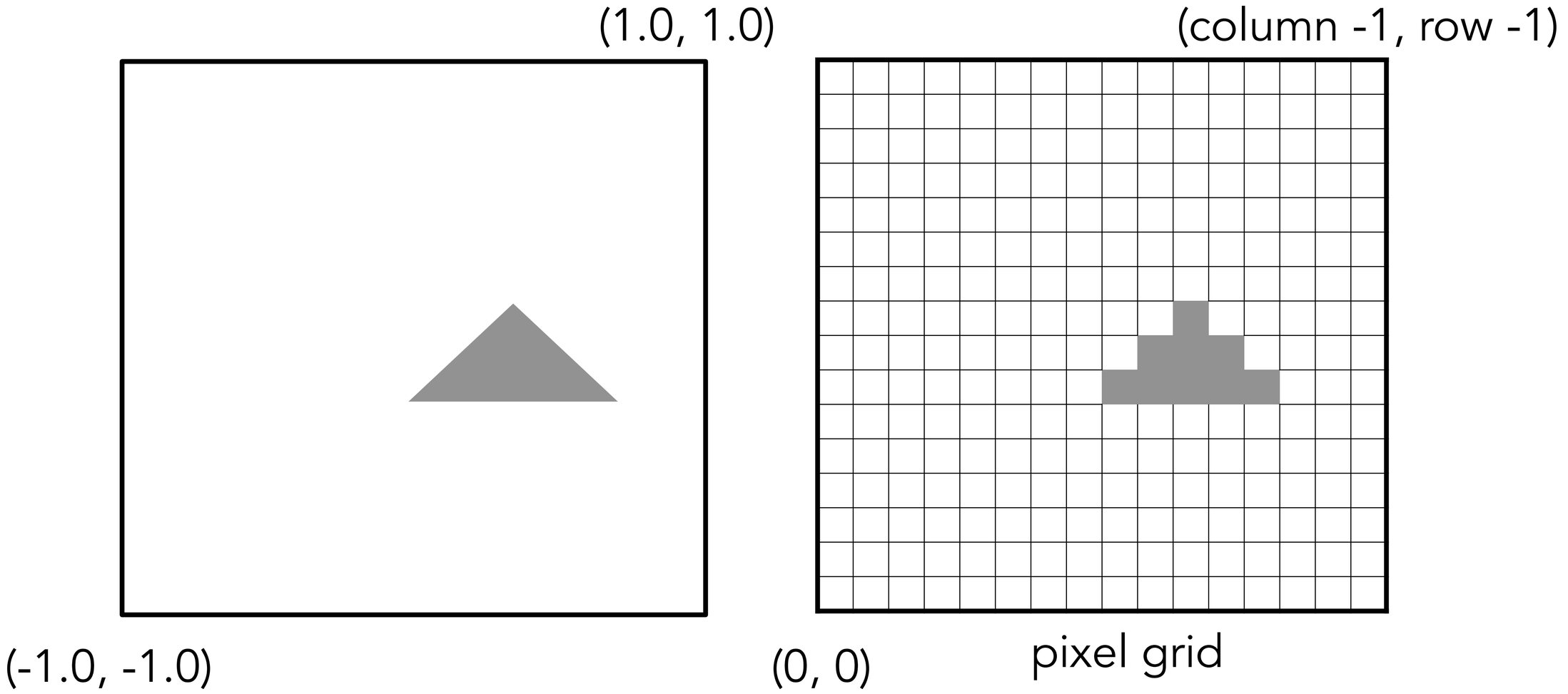
(column -1, row -1)



(0, 0)

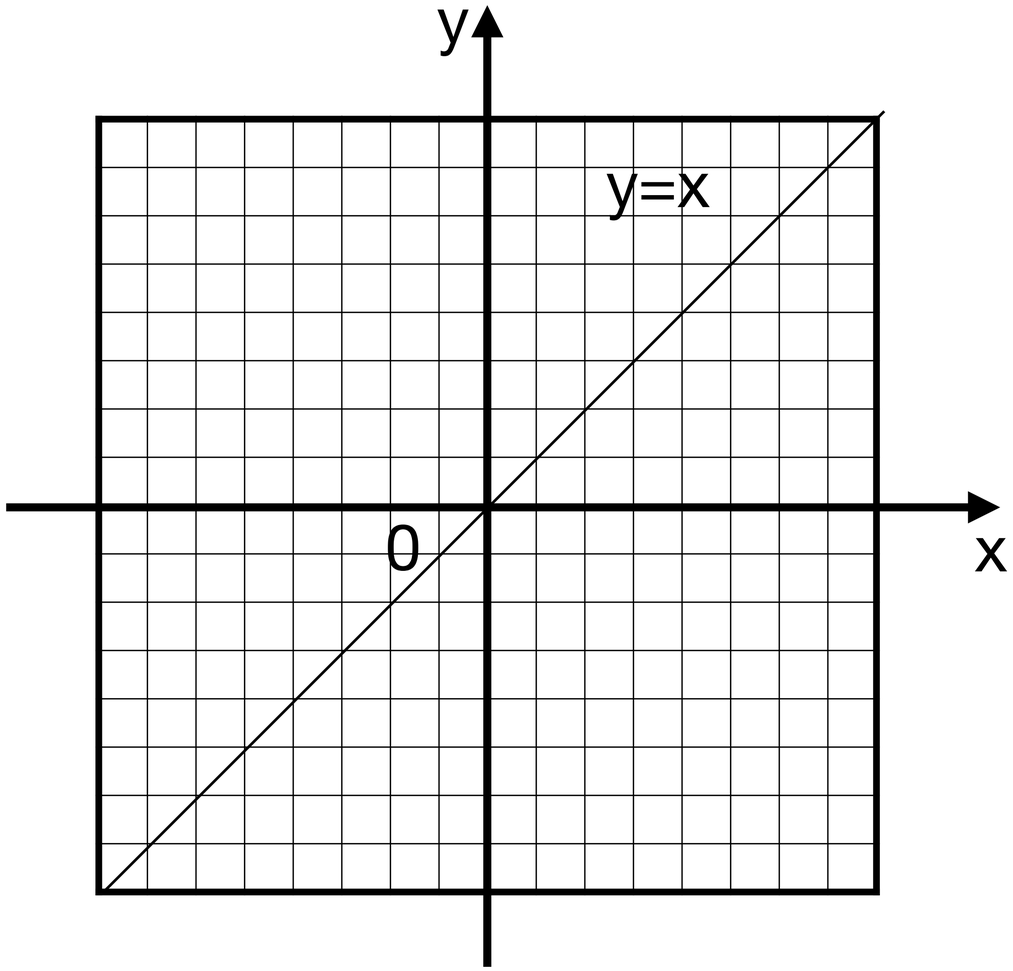
pixel grid

Approximating Shapes with Pixels



Explicit Representation

$y = f(x)$



For each x,

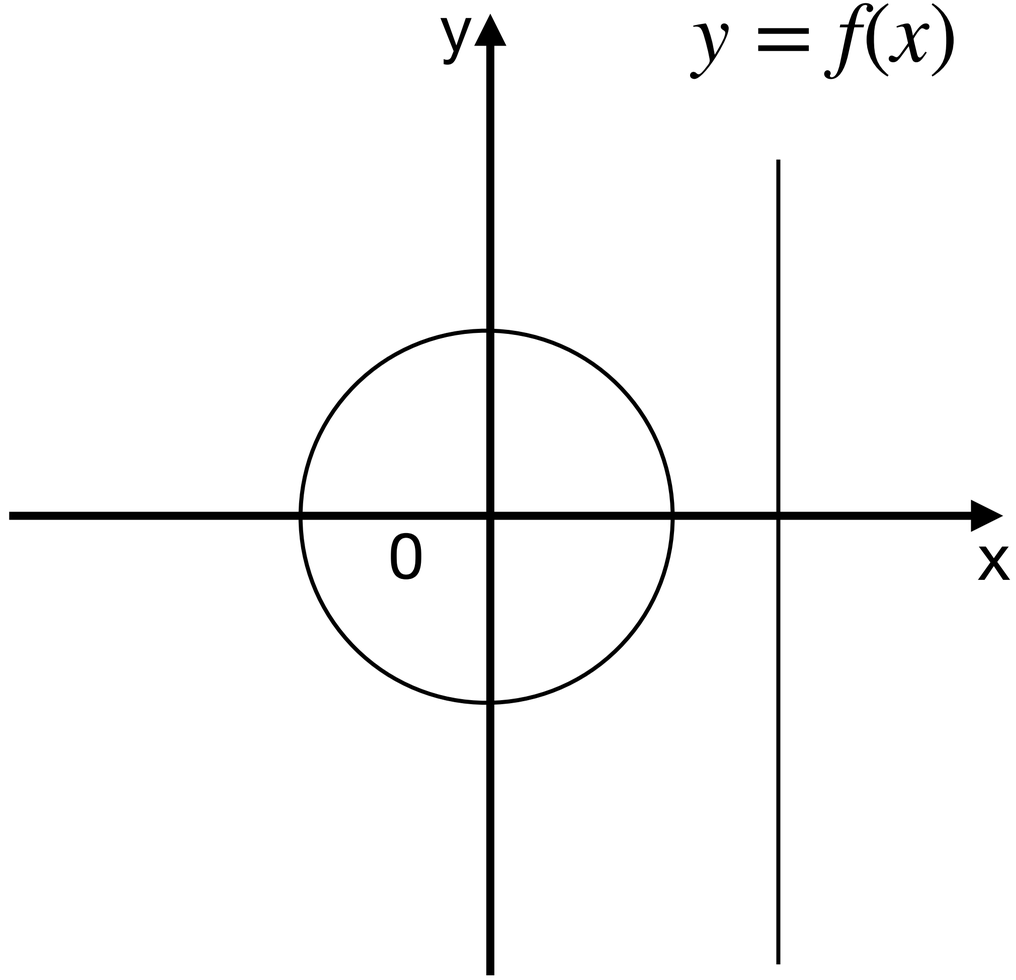
finding the pixels by $\lfloor y \rceil, \quad y = f(x)$

Explicit Representation

Limitations

?

$y = f(x)$



Implicit Representation

$$f(x, y) = 0$$

Lines $f(x, y) = ax + by + c$

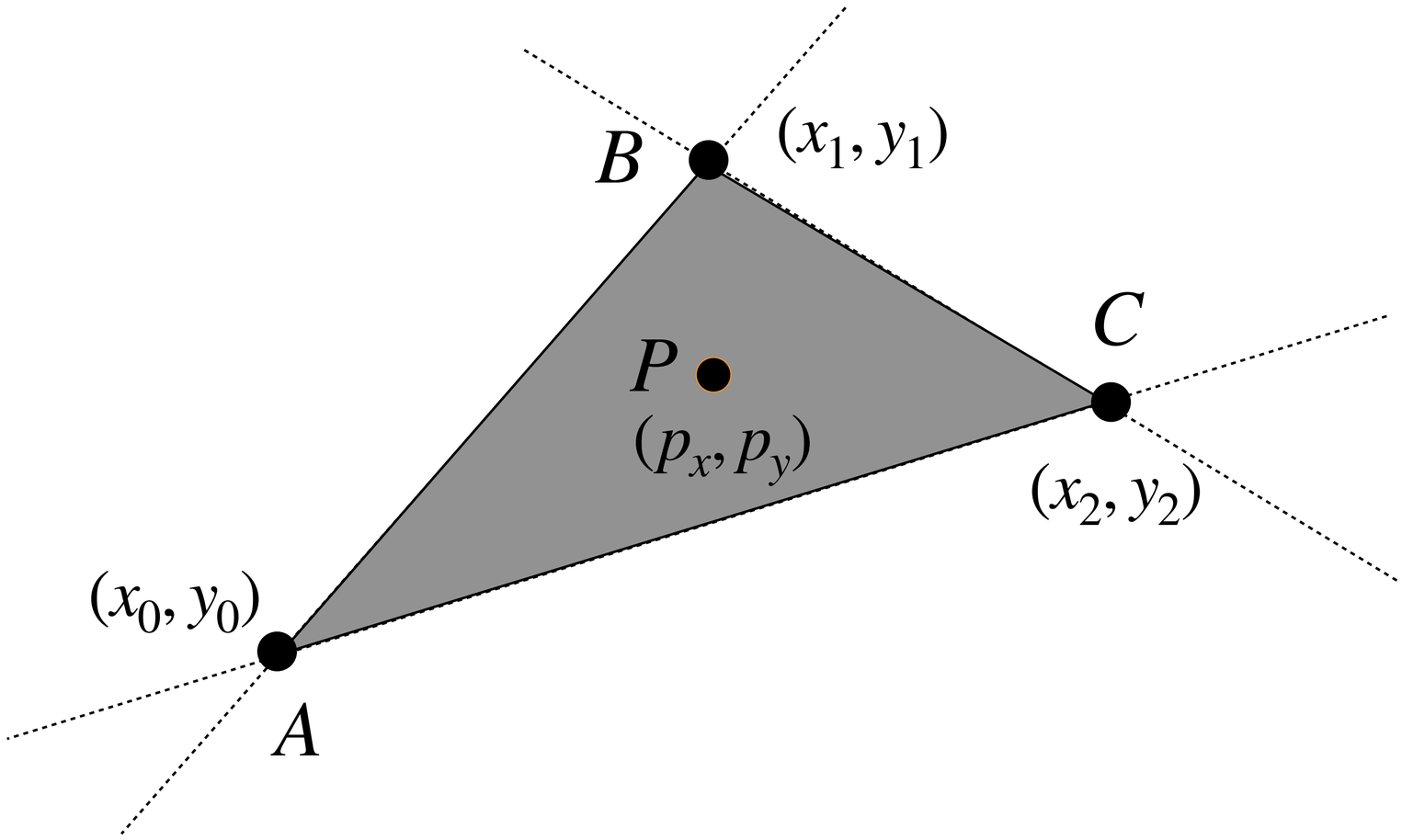
Circles $f(x, y) = (x - c_x)^2 + (y - c_y)^2 - r^2$

On the shape $f(x, y) = 0$

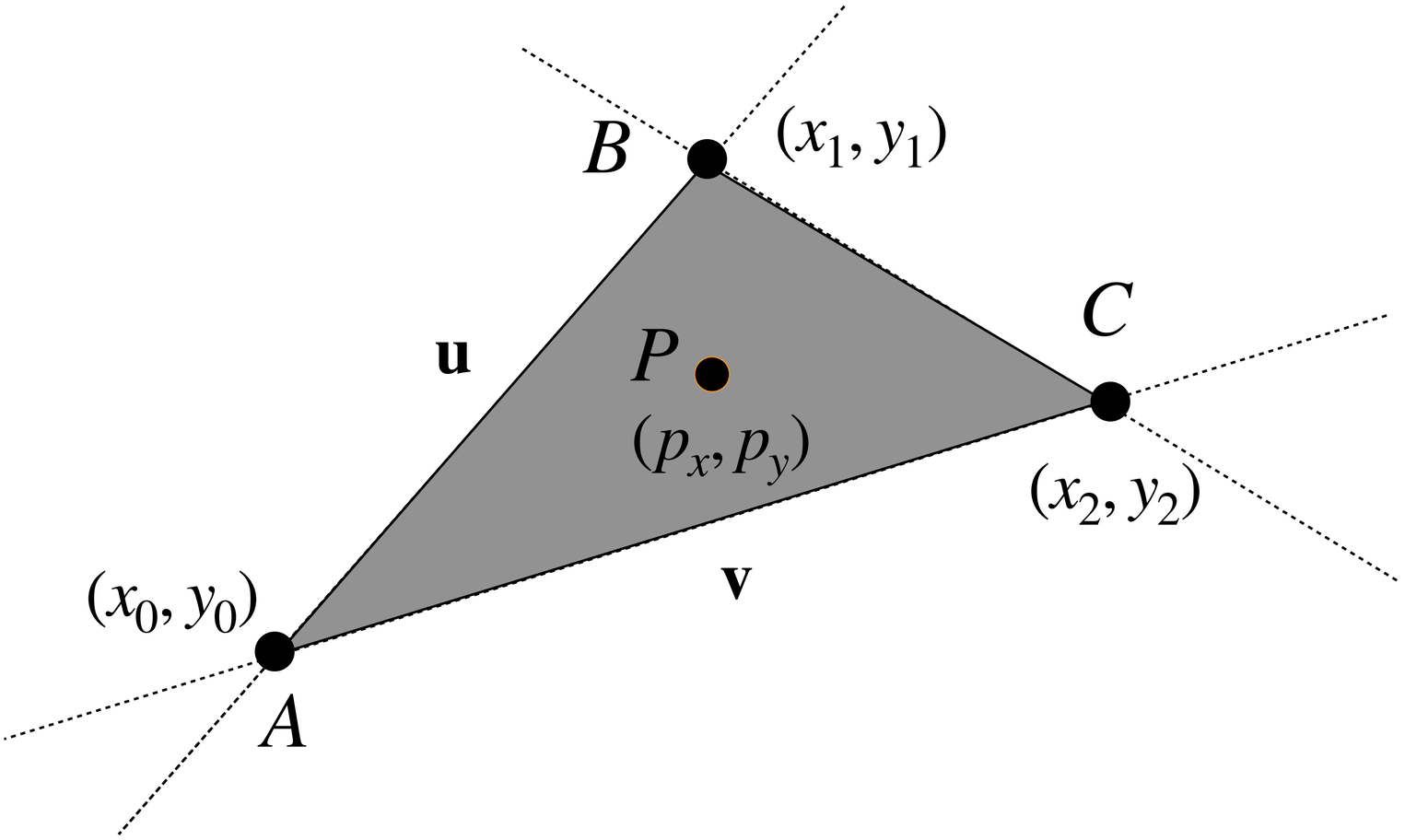
Inside the shape $f(x, y) < 0$

Outside the shape $f(x, y) > 0$

Triangle Rasterization



Barycentric Coordinates



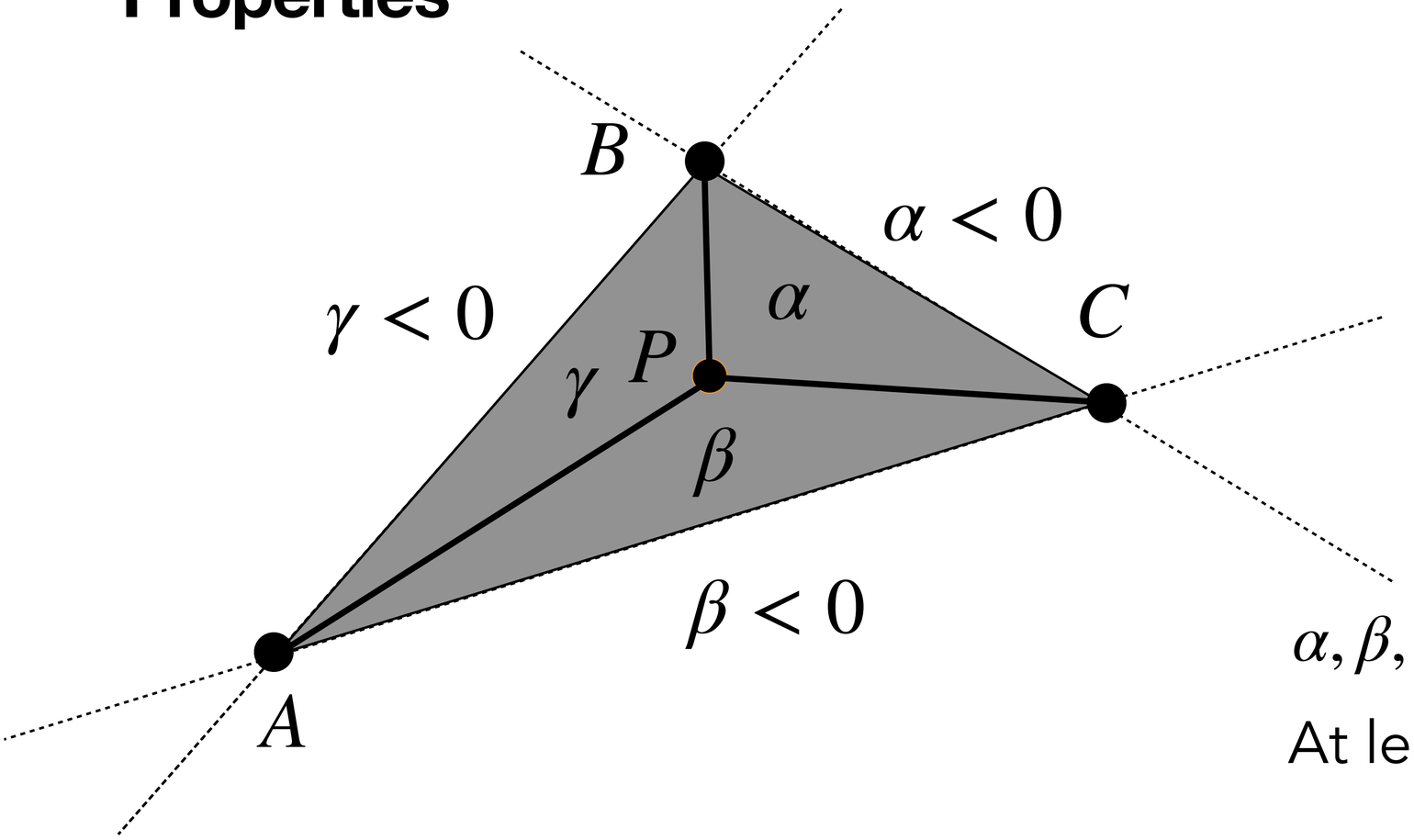
$$\mathbf{p} = A + \mathbf{u}(B - A) + \mathbf{v}(C - A)$$

$$= (1 - \mathbf{u} - \mathbf{v})A + \mathbf{u}B + \mathbf{v}C$$

$$\mathbf{p} = \alpha A + \beta B + \gamma C \quad \text{where} \quad \alpha + \beta + \gamma = 1$$

Barycentric Coordinates

Properties



$$\alpha = \frac{Area(PBC)}{Area(ABC)}$$

$$\beta = \frac{Area(PCA)}{Area(ABC)}$$

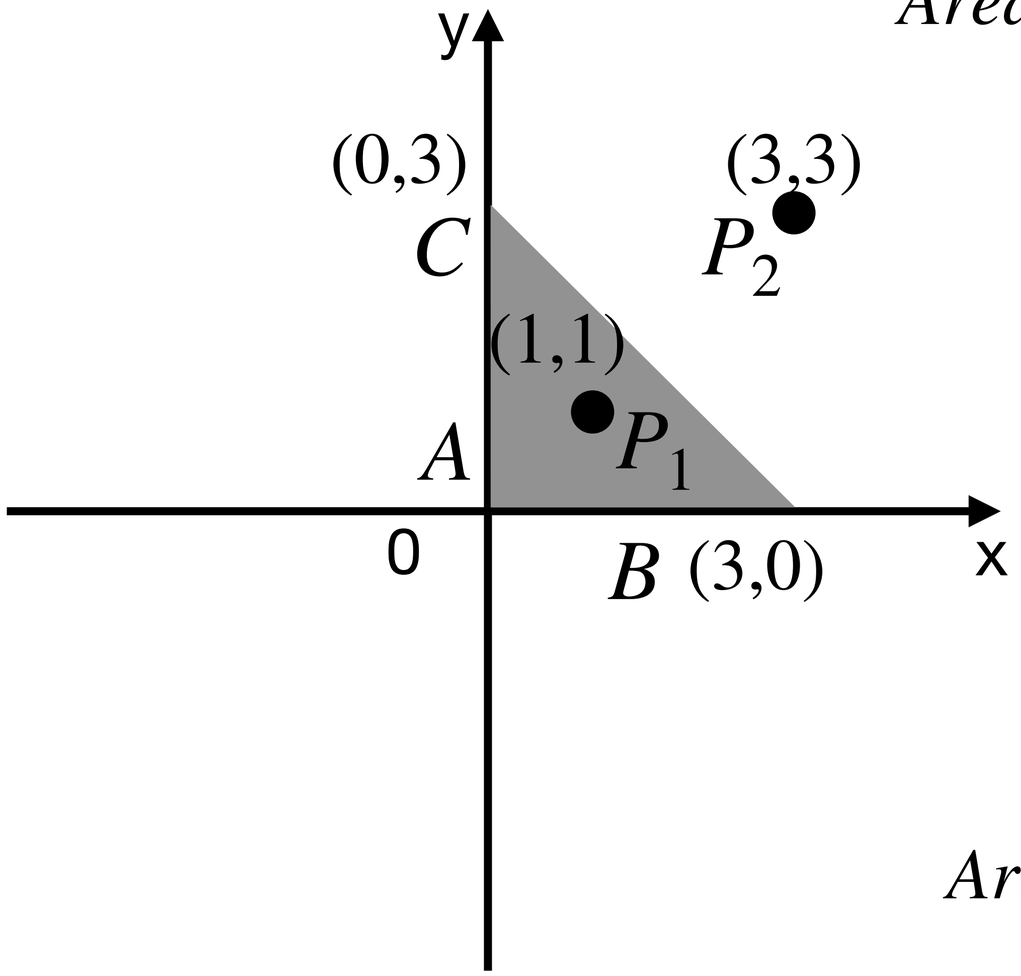
$$\gamma = \frac{Area(PAB)}{Area(ABC)}$$

$\alpha, \beta, \gamma > 0$ if P is inside

At least one of them < 0, if P is outside

$\mathbf{p} = \alpha A + \beta B + \gamma C$ where $\alpha + \beta + \gamma = 1$

Barycentric Coordinates



$$Area(ABC) = \frac{1}{2}\begin{vmatrix} x_A & y_A & 1 \\ x_B & y_B & 1 \\ x_C & y_C & 1 \end{vmatrix} = \frac{1}{2}(B-A)\times(C-A)$$

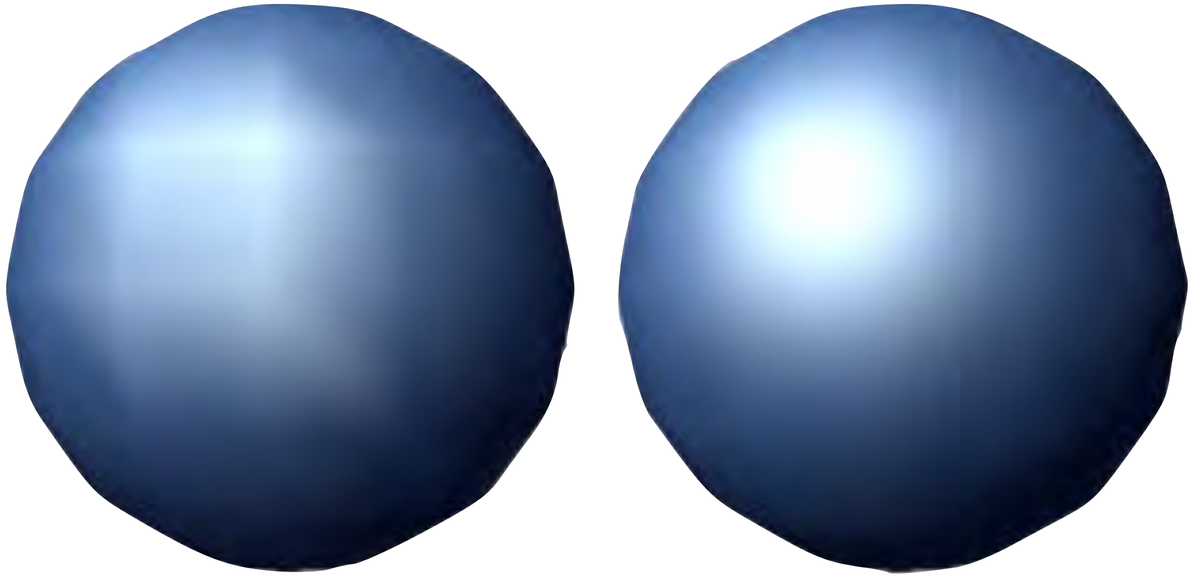
$$= \frac{1}{2}[(x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A)]$$

$$Area(ABC) = \frac{1}{2}\begin{vmatrix} 0 & 0 & 1 \\ 3 & 0 & 1 \\ 0 & 3 & 1 \end{vmatrix} = 4.5$$

$$Area(PBC) = \frac{1}{2}\begin{vmatrix} P_x & P_y & 1 \\ 3 & 0 & 1 \\ 0 & 3 & 1 \end{vmatrix} = \begin{matrix} 1.5 & P_1 : (1,1) \\ -4.5 & P_2 : (3,3) \end{matrix}$$

Barycentric Coordinates

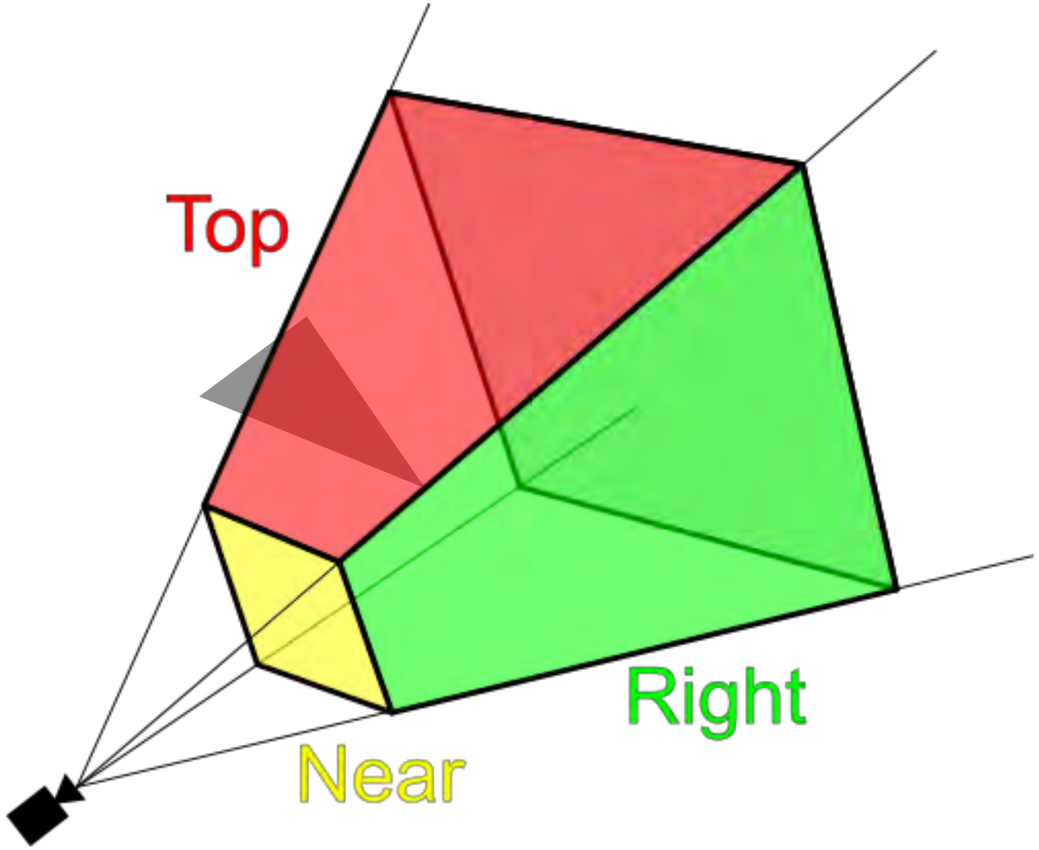
Pixel Interpolations



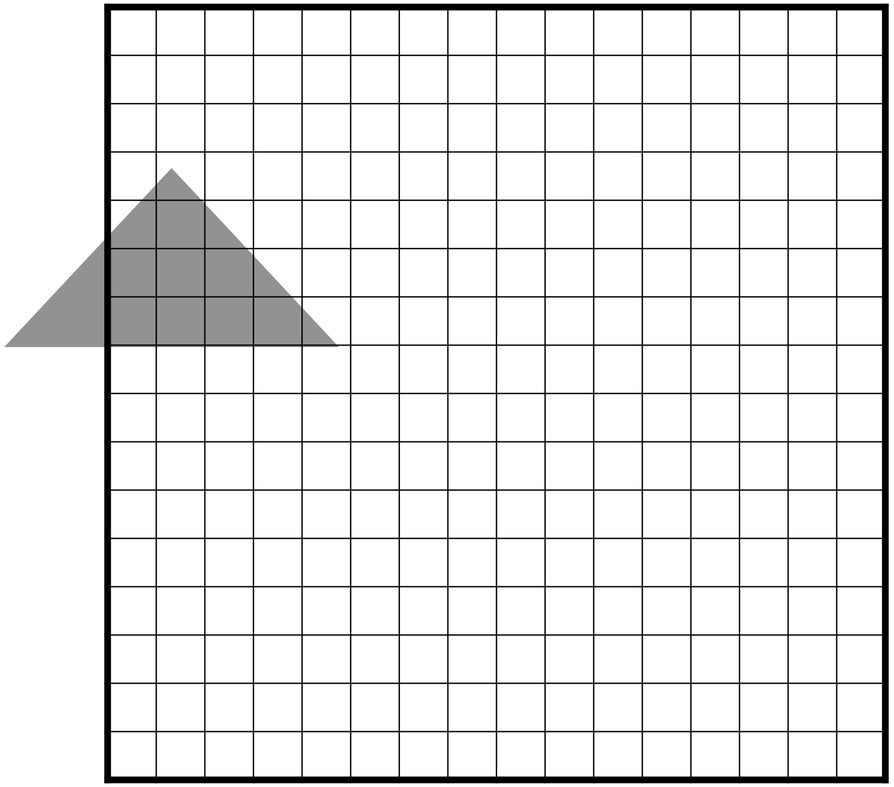
Per-vertex lighting Per-pixel lighting

[source:CGChannel.com]

Clipping

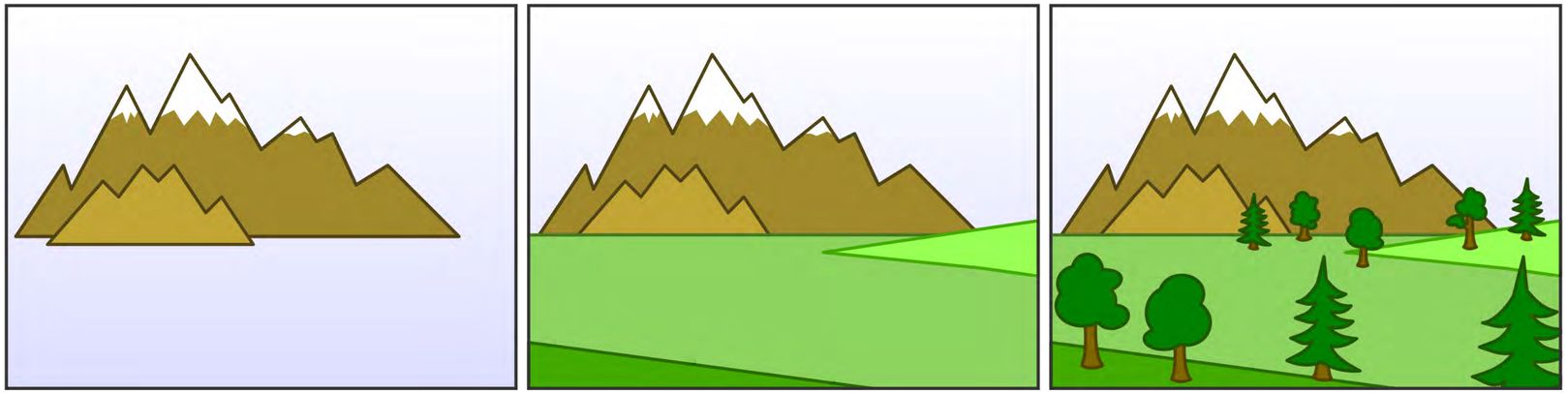


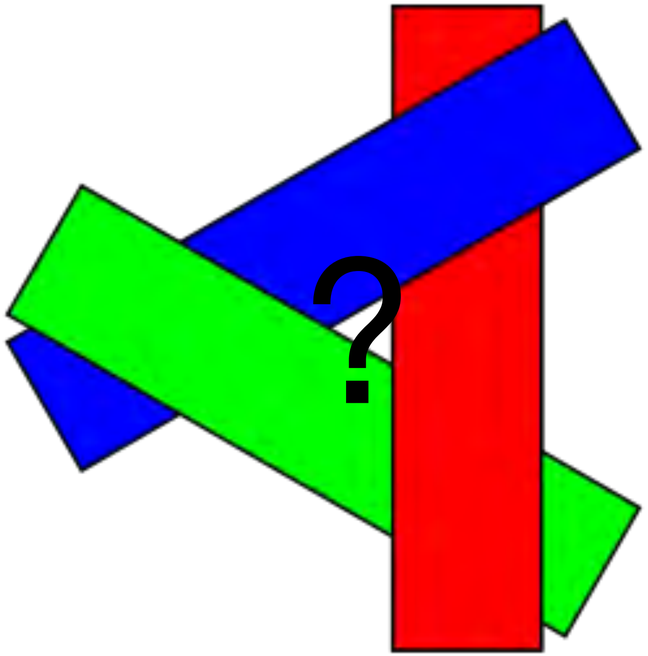
[source:Wikipedia]



Depth Sorting

Painter's Algorithm

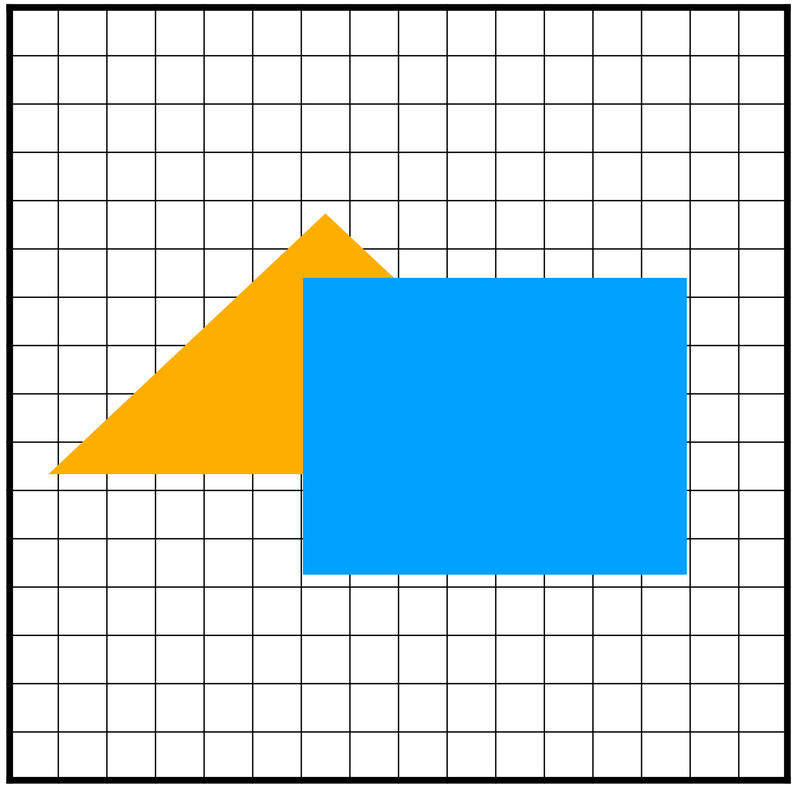


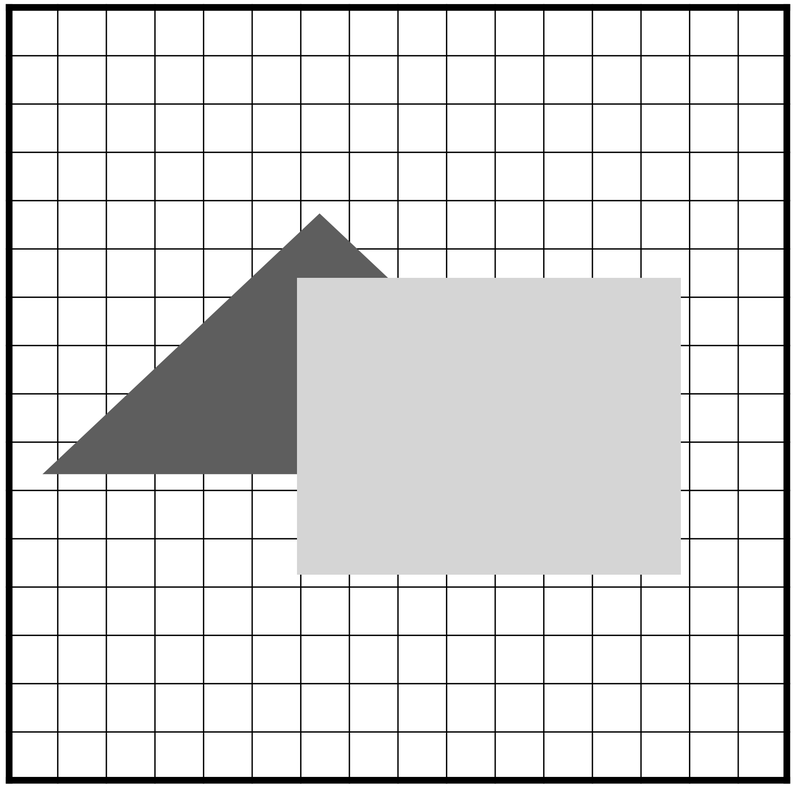


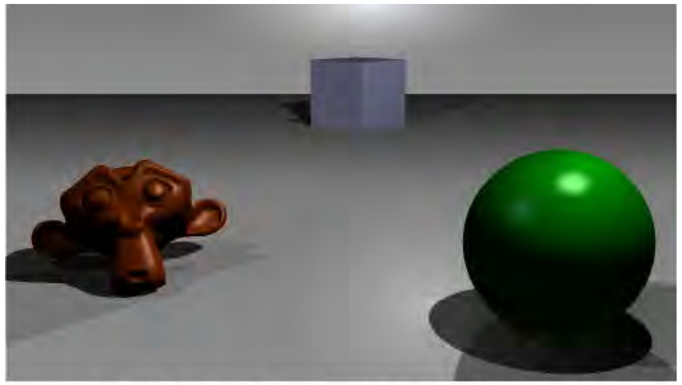
[source:Wikipedia]

Z-buffering

Painter's Algorithm







A simple three-dimensional scene

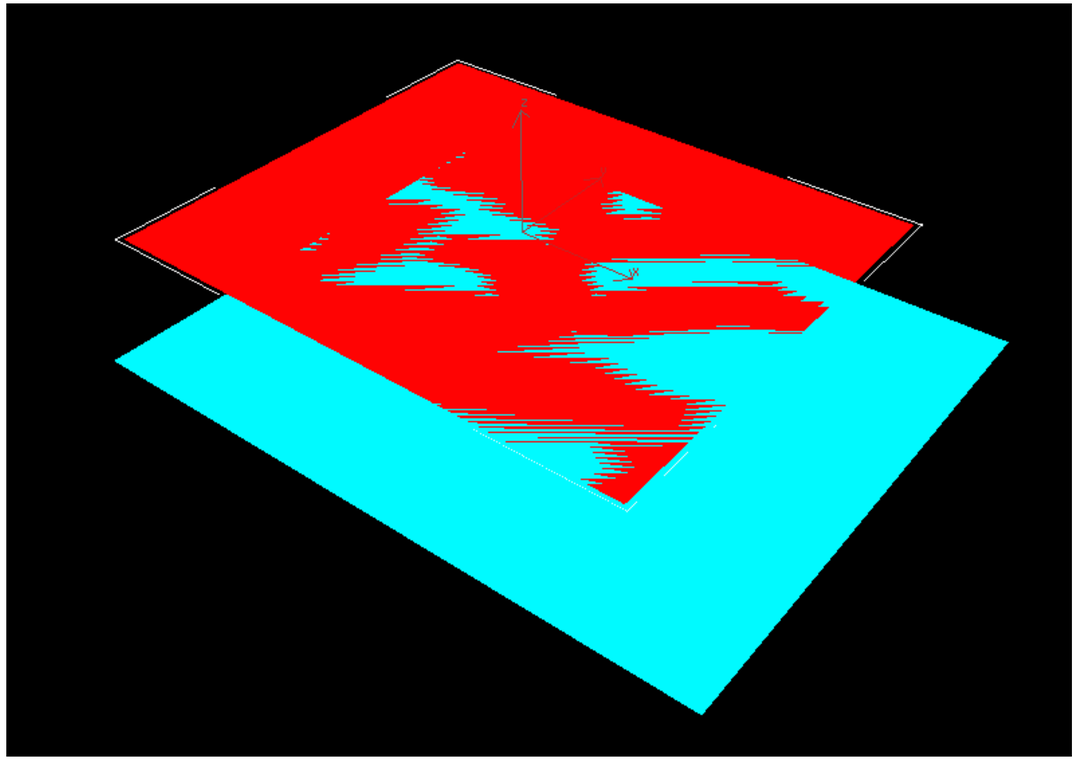


Z-buffer representation

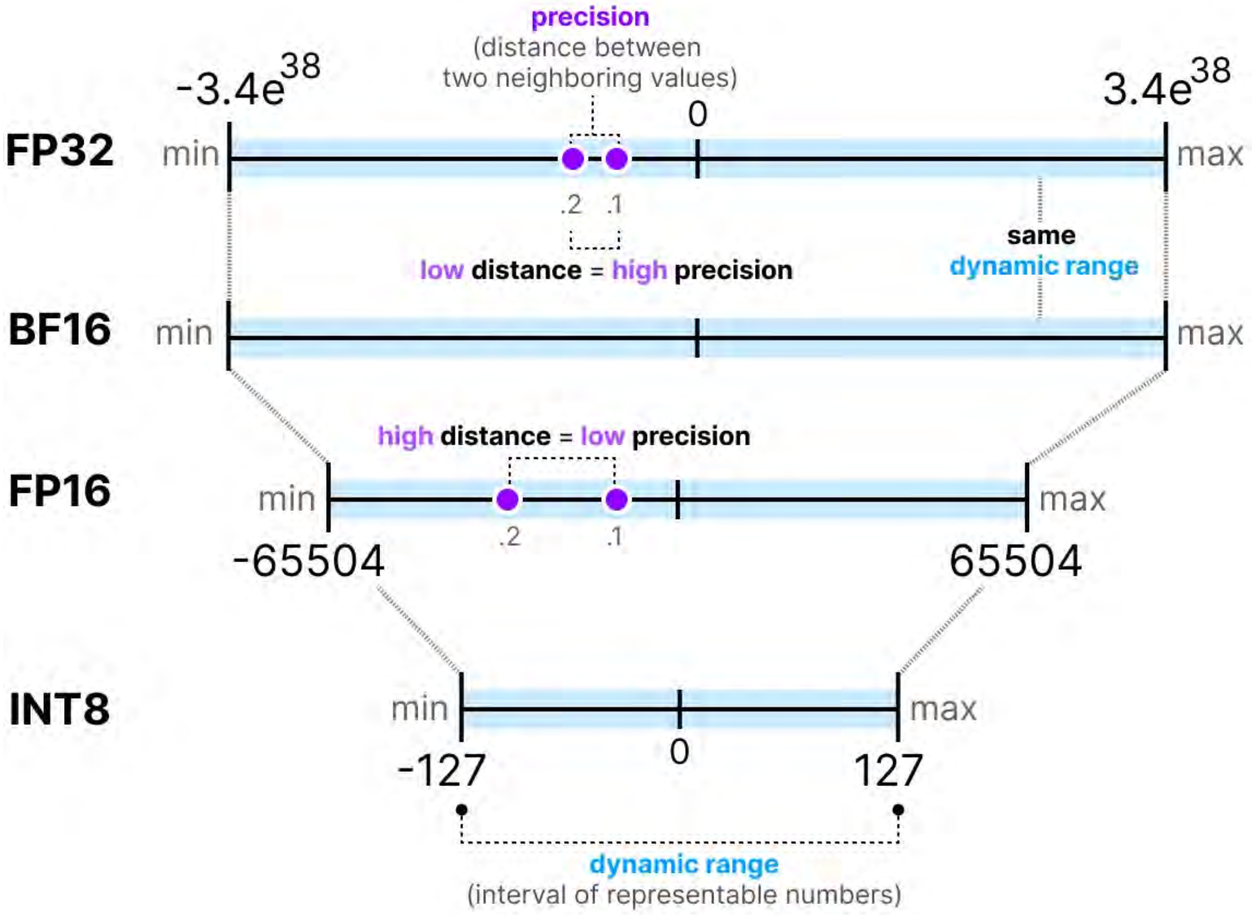
[source:Wikipedia]

Z-buffering

Z-fighting



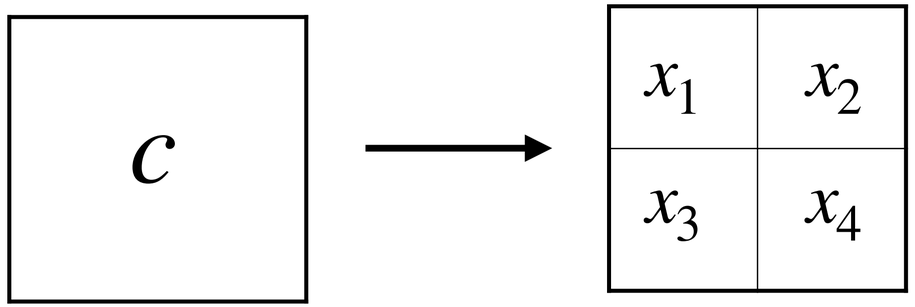
[source:Wikipedia]

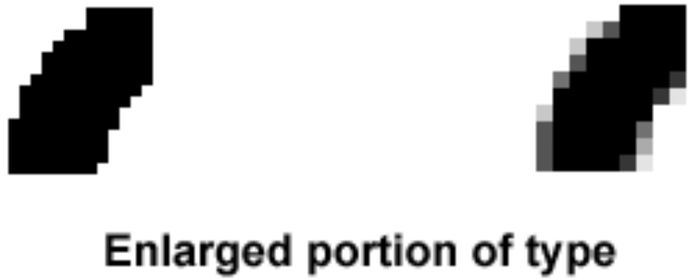


[source:MAARTEN GROOTENDORST]

Anti-aliasing







$$c = \bar{x} = \frac{x_1 + x_2 + x_3 + x_4}{4}$$

[source: Fritz Kessler]

3D Transformation

3D Representation

2D point $(x, y, 1)$ → 3D point $(x, y, z, 1)$

2D vector $[x\ y\ 0]^T$ → 3D vector $[x\ y\ z\ 0]^T$

Homogeneous Coordinates

2D

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix}$$

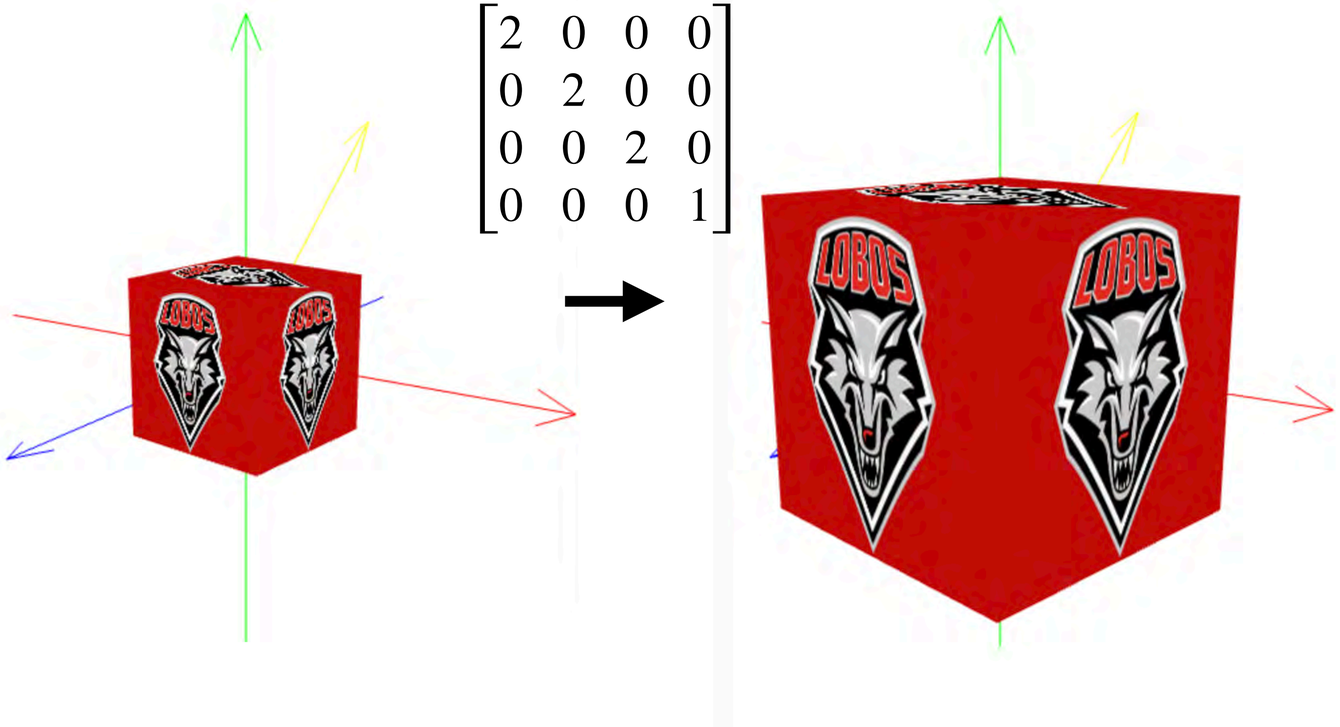
→

3D

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

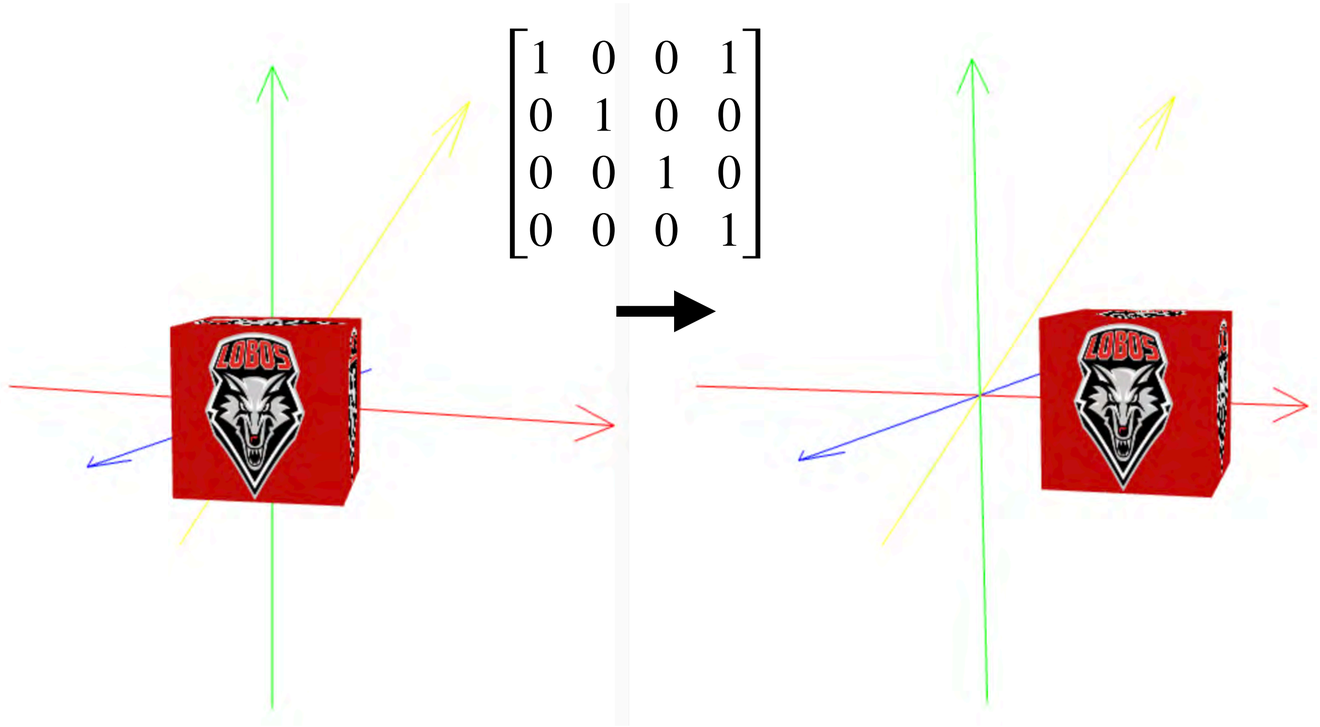
3D Scaling

$$\begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



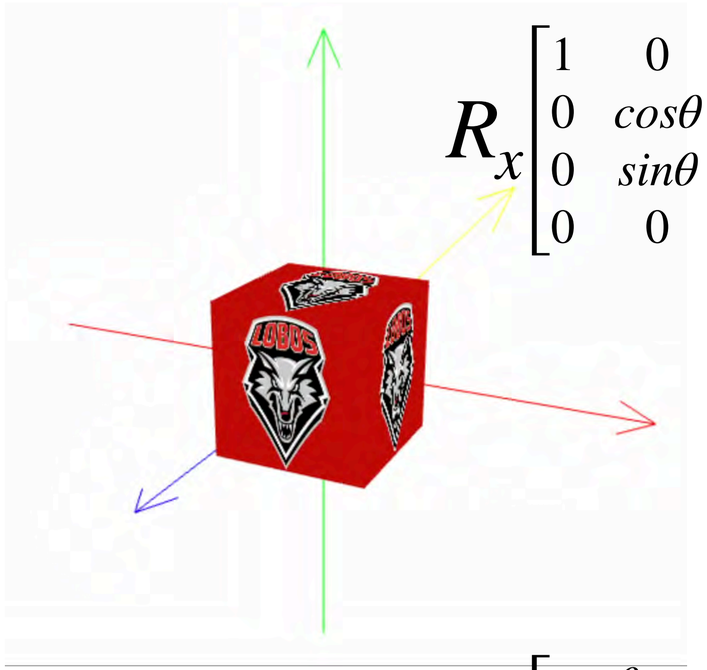
3D Translation

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

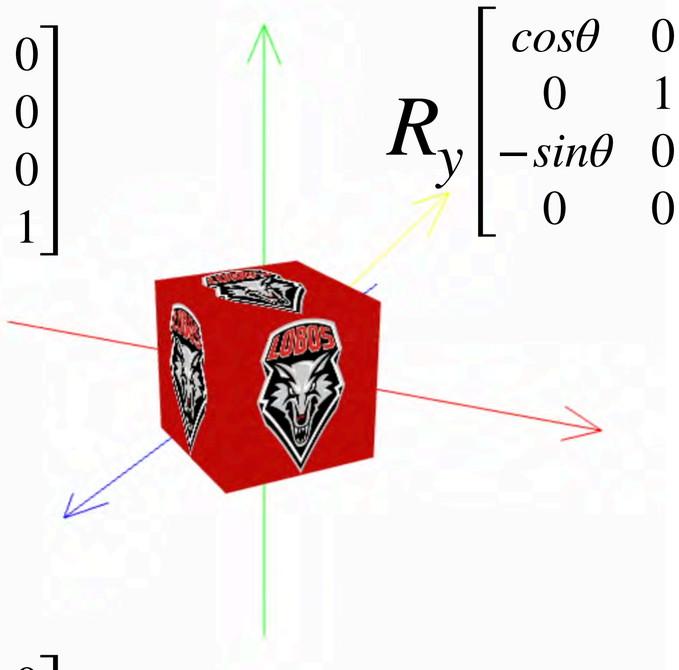


3D Rotation

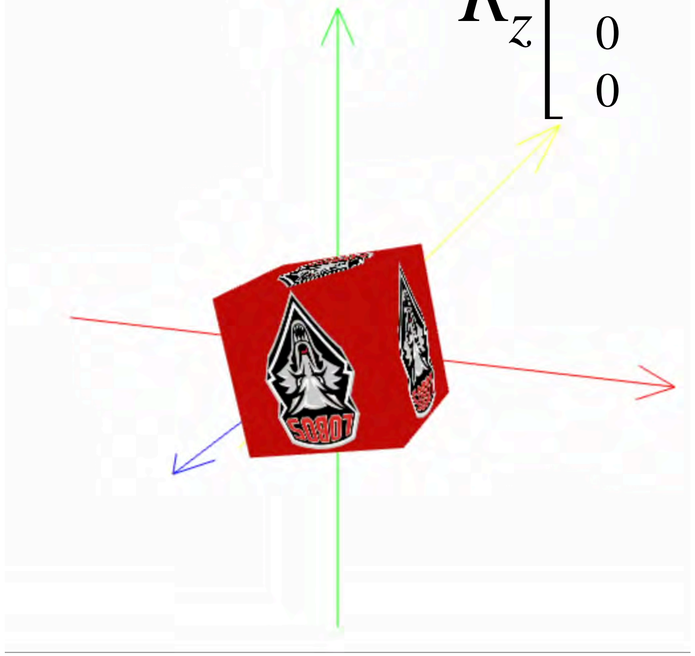
$$\begin{bmatrix} cos\theta & -sin\theta & 0 \\ sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



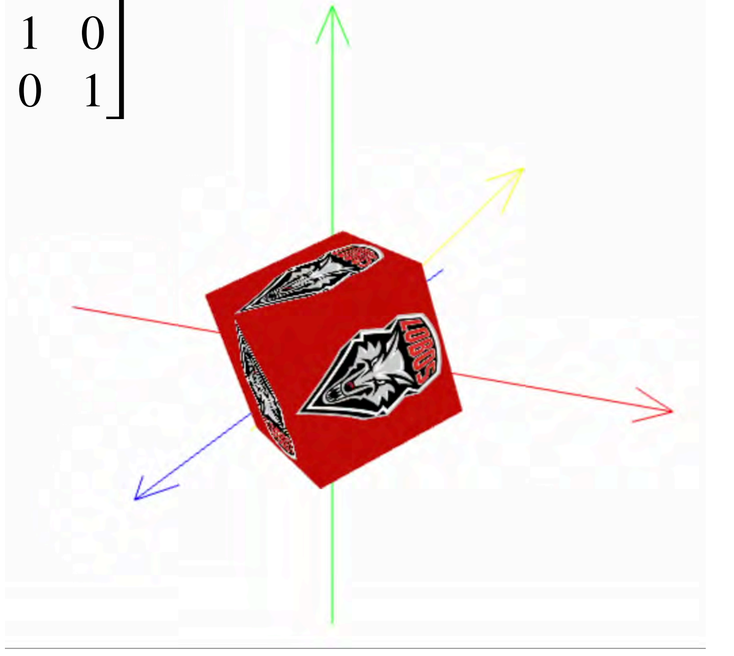
$$R_x \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos\theta & -sin\theta & 0 \\ 0 & sin\theta & cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



$$R_y \begin{bmatrix} cos\theta & 0 & sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -sin\theta & 0 & cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



$$R_z \begin{bmatrix} cos\theta & -sin\theta & 0 & 0 \\ sin\theta & cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



General Rotation

Rodrigues' rotation formula

If rotate θ around an arbitrary unit vector $\mathbf{u} = (u_x, u_y, u_z)$

$$R = \begin{bmatrix} \cos\theta + (1-\cos\theta)u_x^2 & (1-\cos\theta)u_x u_y - u_z \sin\theta & (1-\cos\theta)u_x u_z + u_y \sin\theta \\ (1-\cos\theta)u_x u_y + u_z \sin\theta & \cos\theta + (1-\cos\theta)u_y^2 & (1-\cos\theta)u_y u_z - u_x \sin\theta \\ (1-\cos\theta)u_x u_z - u_y \sin\theta & (1-\cos\theta)u_y u_z + u_x \sin\theta & \cos\theta + (1-\cos\theta)u_z^2 \end{bmatrix}$$

General Rotation

Representation as a sequence of rotations around coordinate axes

$$R = R_z(\gamma)R_y(\beta)R_x(\alpha) \qquad R_z(\gamma)R_y(\beta)R_x(\alpha) \neq R_x(\alpha)R_y(\beta)R_z(\gamma)$$ or other permutations

$$R = \begin{bmatrix} cos\gamma & -sin\gamma & 0 \\ sin\gamma & cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} cos\beta & 0 & sin\beta \\ 0 & 1 & 0 \\ -sin\beta & 0 & cos\beta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & cos\alpha & -sin\alpha \\ 0 & sin\alpha & cos\alpha \end{bmatrix}$$

$$= \begin{bmatrix} cos\gamma\ cos\beta & cos\gamma\ sin\beta\ sin\alpha - sin\gamma\ cos\alpha & cos\gamma\ sin\beta\ cos\alpha + sin\gamma\ sin\alpha \\ sin\gamma\ cos\beta & sin\gamma\ sin\beta\ sin\alpha + cos\gamma\ cos\alpha & sin\gamma\ sin\beta\ cos\alpha - cos\gamma\ sin\alpha \\ -sin\beta & cos\beta\ sin\alpha & cos\beta\ cos\alpha \end{bmatrix}$$

Scaling, Rotation, Translation

The most common pipeline

The final transformation matrix $M = T \cdot R \cdot S$

$$p' = M \cdot p = T \cdot R \cdot S \cdot p$$